

Lignin Python scripts

Ethan Moore

February 2026

1 Introduction

Hi there, if you are reading this, that means you most likely picked up Dr. Wilson's biofilms project from where I left it. In this paper, I will teach you how to use python to automate the analysis of your images. If you are already familiar with python this should be straightforward for you but if not, don't worry, this project was my first dive into python as well. There are 2 scripts I use when working with the images, the first is to rename the images all at once and the second is the actual analysis.

2 Starting with python

If you are already familiar with python, you can skip this section to the next section. If not, welcome to python. To start, you will need to download python to your computer. I personally use a Windows device, but I will make sure to include a tutorial for Apple users if there are differences along the way. To download python, all you need to do is go to python.org. Once there, all you need to do is download the version for your device and follow the instructions given. Once installed, we can begin installation of necessary libraries.

2.1 Libraries

In python there are collections of prewritten code called libraries. These do most of the heavy lifting for most coders. In this project we will need to install 5 libraries as the rest used come packaged with basic installs of python.

2.2 Windows

To install libraries on Windows you will need to use the pip command. On your keyboard press the windows key(win) + r and then type cmd and press enter. Once there, you will type the following commands.

1. `python -m pip install pandas`
2. `python -m pip install numpy`
3. `python -m pip install opencv-python`
4. `python -m pip install matplotlib`
5. `python -m pip install openpyxl`

```
C:\Windows\system32\cmd.exe
Microsoft Windows [Version 10.0.19045.6937]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Ethan>python -m pip install pandas
```

Once installed you are ready to begin running these scripts.

2.3 Mac

To be made

3 Running the scripts

Now that you have everything ready to begin, to run the scripts you will have to have them downloaded. I will leave a flash drive with Dr. Wilson to ensure they remain available. I would recommend saving them to a place you will remember. To run the scripts you can either access their folder and right click and select open in terminal or use the `cd \folder` command in the terminal to navigate to it. Once you have the folder open in the terminal to start the script you need to type the following command "py Rename.images.py".

```
C:\Windows\system32\cmd.exe
Microsoft Windows [Version 10.0.19045.6937]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Ethan>cd documents
C:\Users\Ethan\Documents>py rename_images.py_
```

Notes:

While windows is not case-sensitive, Mac devices are so make sure the name of the file is exactly what you type into the terminal capitals and all.

4 Editing the Code

4.1 Code Editors

If you would like to edit the code first you will need a code editor. You can use notepad but I would recommend a dedicated code editor such as vscode. You can download by going to code.visualstudio.com.

4.2 Renaming_images.py

Overall, this script is relatively straightforward. After starting, it prompts the you to decide which folder to select. Then it asks for the prefix you want. I generally use short prefixes such as 'Ag@AL-1', 'L-1', or 'Control'. It then takes all images from within and will sort them from oldest to newest and names them based on the prefix you decided and what row it was from. The final file name is prefix-n.R_m where n is the column number and m is the row number.

The main thing that will be required to change is the 'row_num'. It should always be an integer(1, 2, 3, ...).

Notes:

If you do not take an equal number of images for each row, Every row except the last will have an equal number and the last row will only be the remaining possible.

4.3 Image_analysis.py

Image_analysis.py first asks what folder all your images are saved in then where you want to save your data. Then it loads each image in grayscale, then filters out any pixel whose intensity falls more than 10 standard deviations from the image's own mean — this removes extreme outlier pixels like sensor noise or blown-out highlights before any real analysis happens. It then applies a Gaussian blur to smooth what's left, followed by adaptive Gaussian thresholding: instead of using one fixed brightness cutoff for the whole image, it looks at each pixel relative to the average brightness of a large (725×725 pixel) neighborhood around it, and converts the pixel to pure white ("biofilm") or black ("background") based on that local comparison. This local approach handles uneven lighting across an image better than a single global cutoff would. Finally, it calculates the percentage of white pixels along with the mean, median, and standard deviation of the original filtered grayscale image, and logs those numbers for every image in the folder.

Notes:

The data is saved in a local .csv file which I have found more convenient than a .xlsx on one drive. When you want the data in a .xlsx you can copy and paste the data that way you can keep a copy to prevent losing it. I usually save the .csv in the same folder as the images to know I already analyzed them. The kernel size must be odd values such as (3,3), (5,5) or, (7,7). Larger kernel sizes are more computationally expensive so they take much longer and may also blur too much.